

Resolución: Encontrar el Índice de Rotación k en un Vector

Este documento presenta el diseño de un algoritmo de Divide y Vencerás para encontrar el índice de rotación k en un vector $v[1 \dots n]$ de enteros distintos en tiempo logarítmico $O(\log n)$.

a) Idea del Algoritmo

El enunciado nos indica que el vector original estaba ordenado de forma creciente pero ha sido rotado k veces ($1 \leq k \leq n - 1$). Esto significa que el vector se divide en dos tramos crecientes bien definidos:

1. El primer tramo: $v[1 \dots k]$ (valores altos, empezando en $v[1]$ hasta el máximo $v[k]$).
2. El segundo tramo: $v[k + 1 \dots n]$ (valores bajos, empezando en el mínimo $v[k + 1]$ hasta $v[n]$).

Como $v[n] < v[1]$, el punto de inflexión donde se "rompe" el orden creciente ocurre exactamente en la frontera entre $v[k]$ y $v[k + 1]$, cumpliéndose que:

$$v[k] > v[k + 1]$$

Para encontrar este índice k en tiempo $O(\log n)$, aplicamos una búsqueda binaria. En cada paso, analizamos el elemento central $v[mid]$ para determinar de qué lado de la inflexión nos encontramos y poder descartar la mitad del vector.

b) Criterios de Descarte

Al evaluar la posición media mid del rango actual de búsqueda $[izq, der]$:

1. **Comprobación directa de éxito:**
 - Si $v[mid] > v[mid + 1]$, entonces mid es exactamente el punto de ruptura k . ¡Lo devolvemos!
 - Si $v[mid] < v[mid - 1]$, entonces el elemento anterior es el punto de ruptura ($k = mid - 1$). ¡Lo devolvemos!
2. **Decisión de descarte:**
 - Si $v[mid] > v[1]$: Esto significa que mid todavía se encuentra en el primer tramo creciente (el tramo de valores grandes). Por lo tanto, el punto de ruptura k tiene que estar obligatoriamente a su derecha.
 - **Decisión:** Descartamos la mitad izquierda y buscamos en $[mid + 1, der]$.
 - Si $v[mid] < v[1]$: Esto significa que mid ya ha cruzado al segundo tramo (el tramo de valores pequeños). Por lo tanto, el punto de ruptura k está a su izquierda.
 - **Decisión:** Descartamos la mitad derecha y buscamos en $[izq, mid - 1]$.

c) Algoritmo en Pseudocódigo

A continuación se presenta el pseudocódigo del algoritmo recursivo utilizando índices base 1:

```

Funcion BuscarInflexion(v, izq, der)
Inicio
    // Caso Base: Si el rango se cruza o se reduce a un elemento, no hay rotación
    Si izq >= der Entonces
        Retornar -1
    FinSi

    // Calcular el punto medio
    mid <- ParteEntera((izq + der) / 2)

    // 1. Comprobaciones de éxito inmediato
    Si v[mid] > v[mid + 1] Entonces
        Retornar mid
    FinSi

    Si v[mid] < v[mid - 1] Entonces
        Retornar mid - 1
    FinSi

    // 2. Descarte de mitades comparando con el extremo izquierdo original v[1]
    Si v[mid] > v[1] Entonces
        // El punto de ruptura está a la derecha
        Retornar BuscarInflexion(v, mid + 1, der)
    Sino
        // El punto de ruptura está a la izquierda
        Retornar BuscarInflexion(v, izq, mid - 1)
    FinSi
FinFuncion

Funcion ObtenerK(v, n)
Inicio
    // Llamada inicial para buscar el punto de inflexión en todo el vector
    Retornar BuscarInflexion(v, 1, n)
FinFuncion

```

d) Análisis de Coste

- **Complejidad Temporal:** $O(\log n)$, ya que el tamaño del intervalo de búsqueda se reduce exactamente a la mitad en cada paso recursivo mediante comparaciones elementales de tiempo constante $O(1)$.
- **Complejidad Espacial:** $O(\log n)$ debido al espacio reservado en la pila de llamadas de la recursión, cuya profundidad máxima es proporcional a la altura del árbol de división recursiva ($\log_2 n$).